

Computer Networks Protocol Stack Simulator

1. Project Agenda

This project is a Computer Networks protocol stack simulator. The main goal is to show how data moves from one end device to another through different layers of a network.

The simulator demonstrates Physical, Data Link, Network, Transport, and Application layer concepts using simple Python classes. It is not sending real network traffic. Instead, it prints clear logs to show what would happen in a real network.

The project is designed for learning. Each layer adds its own information to the data while sending, and each layer checks/removes that information while receiving.

2. Modular File Structure

main.py: Entry point of the project. It calls the simulation runner.

network/simulator.py: Builds all test topologies and starts communication between devices.

devices/end_device.py: Represents a PC or server. It contains application, transport, network, data link, and receiving-side logic.

devices/hub.py: Represents a hub. It broadcasts incoming frames to all connected devices.

devices/switch.py: Represents a switch. It performs MAC learning, flooding, and unicast forwarding.

devices/router.py: Represents a router. It handles interfaces, routing tables, static routing, RIP using Bellman-Ford, TTL, and packet forwarding.

models/segment.py: Builds transport layer segments.

models/packet.py: Builds network layer datagrams.

models/frame.py: Builds data link layer frames and attaches CRC.

protocols/checksum.py: Implements CRC-32 error checking.

protocols/csma_cd.py: Implements CSMA/CD access control simulation.

protocols/go_back_n.py: Implements Go-Back-N helper logic such as ACK simulation and message chunking.

utils/logger.py: Prints readable layer-by-layer logs and generates fake MAC addresses.

3. Physical Layer

The Physical layer is shown when devices send and receive bits over a medium. In the simulator, this is represented by logs such as 'Sending bits over medium' and 'Bits Received'.

End devices can be connected directly to each other, to a hub, or to a switch. A hub behaves like a Physical layer device because it does not understand MAC addresses. It simply repeats the signal to all other ports.

Test cases include a dedicated connection between two end devices and a star topology with five end devices connected to one hub.

4. Data Link Layer

The Data Link layer uses MAC addresses to deliver frames inside a local network. Frames are created in models/frame.py and contain source MAC, destination MAC, payload, and CRC.

CRC-32 is used as the error control protocol. The sender calculates CRC before sending, and the receiver recalculates CRC after receiving. If values do not match, the frame is dropped.

CSMA/CD is used as the access control protocol. Before sending, the device checks if the channel is free. If a collision is simulated, the device sends a jam signal, waits using backoff, and tries again.

The switch performs MAC learning. When it receives a frame, it learns the source MAC address and remembers which device/port it came from. If the destination MAC is known, it unicasts the frame. If unknown, it floods the frame.

The simulator reports broadcast and collision domains for switch and hub topologies.

5. Network Layer

The Network layer is responsible for IP addressing and routing. End devices store their IP address and subnet. Routers store interfaces and routing tables.

When an end device sends data, it checks if the destination IP is inside its own subnet. If yes, it sends directly. If no, it sends the frame to the default gateway.

ARP is simulated using an ARP table. If the target IP is found in the ARP table, the device gets the matching MAC address. If not found, the broadcast MAC address ff:ff:ff:ff:ff:ff is

used.

Routers use TTL to prevent packets from travelling forever. Every time a router forwards a packet, it decreases TTL by one. If TTL becomes too low, the packet is dropped.

Routing uses longest mask matching. This means the router chooses the most specific matching route for the destination IP address.

6. Static Routing and RIP

Static routing is implemented by manually adding a route to the router routing table. This is useful when the path is known in advance.

RIP dynamic routing is implemented using the Bellman-Ford distance-vector idea. Routers know their directly connected networks first. Then they exchange route information with RIP neighbors.

In Bellman-Ford, each router checks whether going through a neighbor gives a lower metric to reach a network. If the new metric is better, the routing table is updated.

RIP uses hop count as the metric. A metric of 16 is treated as unreachable. In this simulator, the RIP demo uses three routers and shows that a cheaper path through an intermediate router is selected over a more expensive direct path.

7. Transport Layer

The Transport layer is mainly implemented in `devices/end_device.py` and `protocols/go_back_n.py`.

The simulator uses port numbers for process-to-process communication. Port 80 is used for HTTP, and port 21 is used for FTP. The sender also generates an ephemeral source port between 49152 and 65535.

Go-Back-N is implemented as the sliding window protocol. Data is split into chunks, each chunk becomes a segment, and each segment gets a sequence number.

The sender keeps sent segments in a sender window. If an ACK is lost, the sender retransmits from the missing sequence number. The receiver accepts packets only in the expected order.

The receiver tracks expected sequence numbers per conversation using source IP, source port, and destination port. This allows a new HTTP or FTP request to start again at sequence number 0 without being treated as an old duplicate.

8. Application Layer

The Application layer is implemented in `devices/end_device.py`.

Two application services are simulated: HTTP and FTP. HTTP runs on port 80 and returns 'HTTP 200 OK'. FTP runs on port 21 and returns 'FTP FILE SENT'.

When the receiver gets a valid segment, it checks the destination port. If the port is 80, the HTTP handler is called. If the port is 21, the FTP handler is called. If the port is unknown, the simulator logs 'Port unreachable'.

9. Encapsulation and Decapsulation Flow

Sending flow: Application data is given to the Transport layer. Transport creates a segment with ports and sequence number. Network creates a datagram with source and destination IP. Data Link creates a frame with source and destination MAC plus CRC. Physical layer sends the bits.

Receiving flow: The receiver checks the destination MAC, verifies CRC, checks the destination IP, checks sequence number, checks destination port, and finally delivers the payload to HTTP or FTP.

In simple words, sending wraps the data layer by layer. Receiving unwraps the data layer by layer.

10. Test Cases Implemented

Test Case 1: Two end devices connected directly. This demonstrates basic end-to-end data transmission.

Test Case 2: Five end devices connected to a hub. This demonstrates hub broadcasting and one collision domain.

Test Case 3: Five end devices connected to a switch. This demonstrates MAC learning, flooding, unicast forwarding, CRC, CSMA/CD, and Go-Back-N.

Test Case 4: Two hub-based star topologies connected through one switch. This demonstrates a larger LAN and reports broadcast/collision domains.

Test Case 5: Full stack routing between two subnets using a router. This demonstrates IP addressing, gateway usage, ARP, routing, HTTP, FTP, ports, and complete encapsulation.

Test Case 6: RIP Bellman-Ford dynamic routing demo with three routers. This demonstrates how

routers learn better routes dynamically using distance-vector updates.

11. Current Limitations

The simulator is educational and simplified. It does not send real packets on a real network.

ARP is simulated using table lookup and broadcast fallback. It does not implement a full ARP request/reply exchange.

ACKs are simulated locally for Go-Back-N instead of travelling back through the entire network path.

RIP is implemented with Bellman-Ford route convergence, but it does not simulate timed periodic updates like a real router would.

A bridge device is not currently implemented. The switch covers Layer 2 learning and forwarding behavior.

12. Conclusion

This project successfully demonstrates how a network protocol stack works from top to bottom. It shows how data is created by an application, segmented by the Transport layer, addressed by the Network layer, framed by the Data Link layer, and sent through the Physical layer.

The simulator also demonstrates important networking concepts such as hubs, switches, MAC learning, CRC, CSMA/CD, Go-Back-N, IPv4 subnets, ARP, static routing, RIP with Bellman-Ford, HTTP, and FTP.